

simple-plot

Typst Package

Mathematical Function Plotting

A lightweight library for creating elegant mathematical plots

Version 0.7.0

Nathan Scheinmann

Contents

1. Introduction	3
1.1. Features	3
1.2. Installation	3
1.3. Quick Start	3
2. Basic Usage	4
2.1. Plotting Functions	4
2.2. Mathematical Functions Reference	4
2.3. Function Domain	6
2.4. Function Labels	7
2.5. Data Points and Scatter Plots	7
3. Grid Options	9
3.1. Grid Modes	9
3.2. Minor Grid Subdivisions	9
3.3. Grid Label Break	11
4. Axis Configuration	12
4.1. Axis Position	12
4.2. Axis Extension	12
5. Tick Configuration	13
5.1. Default Integer Ticks	13
5.2. Custom Tick Step	13
5.3. Tick Label Step	15
5.4. Unit Label Only	15
5.5. Custom Tick Positions	17
5.6. Hide Origin Label	17
6. Markers	18
6.1. Available Marker Types	18
6.2. Using Markers	19
7. Convenience Functions	20
7.1. <code>plot-fn</code> - Quick Function Plot	20
7.2. <code>scatter</code> - Scatter Plot Helper	20
7.3. <code>line-plot</code> - Line Plot Helper	20
7.4. <code>func-plot</code> - Function Plot Helper	20
7.5. <code>riemann-sum</code> - Riemann Sum Rectangles	22
7.6. <code>volume-of-revolution</code> - Volume of revolution diagram	24
8. Global Configuration	26
8.1. Setting Defaults	26
8.2. Resetting Defaults	26
9. Styling	27
9.1. Custom Styles	27
9.2. Default Style Values	27
10. Parameter Reference	28
10.1. <code>plot</code> Function	28
10.2. Function/Data Specification	30
10.3. <code>riemann-sum</code> Function	31
10.4. <code>volume-of-revolution</code> Function	32
11. Complete Examples	33
11.1. Trigonometric Functions	33
11.2. Polynomial with Fine Grid	34
11.3. Minimal Style Plot	35
11.4. Data with Trend Line	36

1. Introduction

`simple-plot` is a Typst package for creating clean, elegant mathematical plots. Built on CeTZ, it provides an intuitive interface for plotting functions, data points, and creating publication-ready graphs.

1.1. Features

- Plot mathematical functions with automatic sampling
- Scatter plots and line plots with customizable markers
- Clean integer-based tick system by default
- Major and minor grid with elegant styling
- Gap-based grid line breaks around tick labels (`grid-label-break`)
- Automatic axis extension beyond grid
- Flexible axis positioning (origin, bottom/left, custom)
- Multiple label display options (unit-label-only, label-step)
- Function labels with flexible positioning
- Riemann sum rectangles with left, right, and midpoint methods
- Volume of revolution diagrams with end caps, disks, and tilted axes
- Clipping for clean rendering at boundaries

1.2. Installation

Import the package in your Typst document:

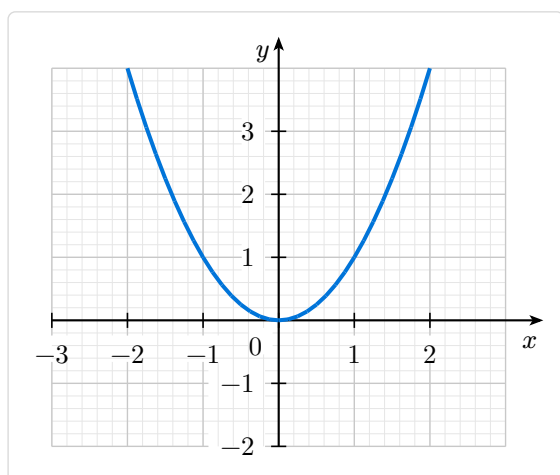
```
#import "@preview/simple-plot:0.8.0": plot
```

1.3. Quick Start

Code

```
#plot(  
  width: 6, height: 5,  
  xmin: -3, xmax: 3, ymin: -2, ymax: 4,  
  xlabel:  $x$ , ylabel:  $y$ ,  
  show-grid: true,  
  (fn: x => x * x, stroke: blue + 1.5pt),  
)
```

Preview



2. Basic Usage

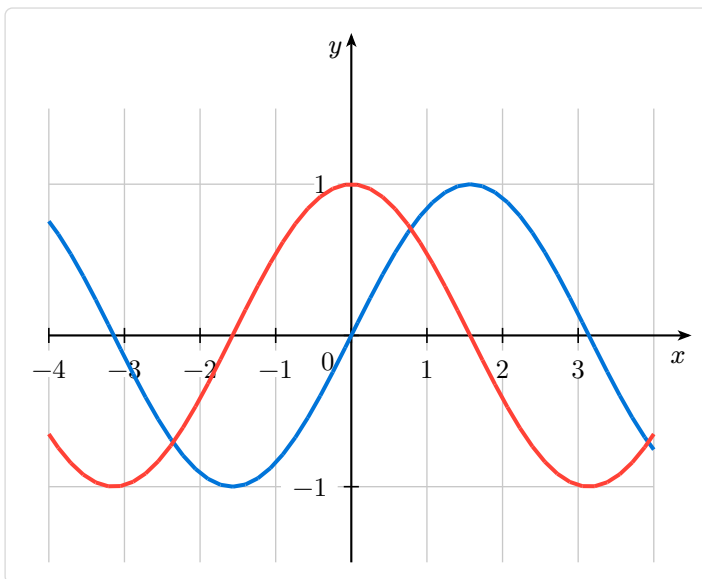
2.1. Plotting Functions

Plot mathematical functions by passing a dictionary with `fn`:

Code

```
#plot(  
  width: 8, height: 6,  
  xmin: -4, xmax: 4, ymin: -1.5, ymax: 1.5,  
  xlabel:  $x$ , ylabel:  $y$ ,  
  show-grid: "major",  
  (fn: x => calc.sin(x), stroke: blue + 1.5pt),  
  (fn: x => calc.cos(x), stroke: red + 1.5pt),  
)
```

Preview



2.2. Mathematical Functions Reference

Functions are defined using Typst's `calc` module. Here are the most common mathematical functions:

Function	Typst syntax
Power x^n	<code>calc.pow(x, n)</code>
Square root $\sqrt{x}$	<code>calc.sqrt(x)</code>
Absolute value $ x $	<code>calc.abs(x)</code>
Sine $\sin(x)$	<code>calc.sin(x)</code>
Cosine $\cos(x)$	<code>calc.cos(x)</code>
Tangent $\tan(x)$	<code>calc.tan(x)</code>
Exponential e^x	<code>calc.exp(x)</code>
Natural log $\ln(x)$	<code>calc.ln(x)</code>
Log base b	<code>calc.log(x, base: b)</code>
Maximum	<code>calc.max(a, b)</code>
Minimum	<code>calc.min(a, b)</code>

Important: When using constants in calculations, use decimal notation (e.g., 2.0 instead of 2) to avoid type errors. For example:

- ✓ $x \Rightarrow x * x / 2.0$
- ✗ $x \Rightarrow x * x / 2$ (*may cause errors*)

This is because Typst's type system requires consistent float arithmetic.

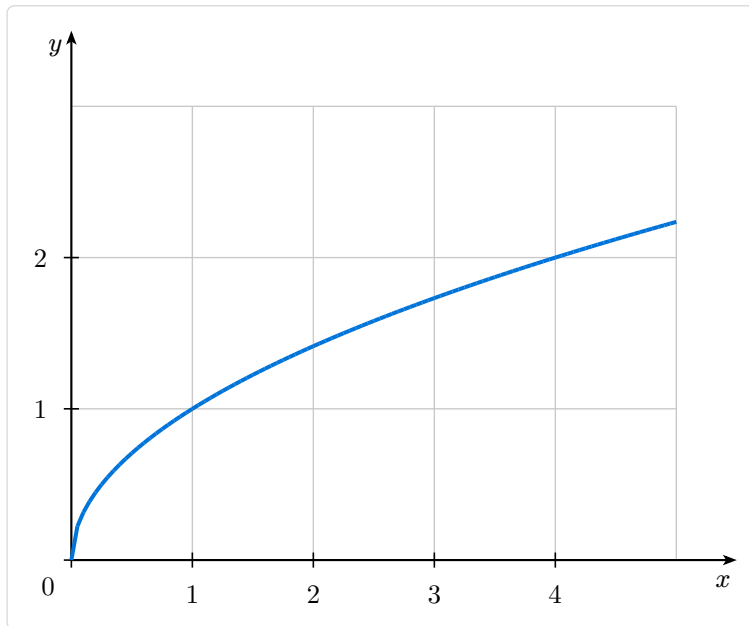
2.3. Function Domain

Specify a custom domain for functions:

Code

```
#plot(  
  width: 8, height: 6,  
  xmin: 0, xmax: 5, ymin: 0, ymax: 3,  
  xlabel:  $x$ , ylabel:  $y$ ,  
  axis-x-pos: "bottom", axis-y-pos: "left",  
  show-grid: "major",  
  (fn: x => calc.sqrt(x), domain: (0, 5), stroke: blue + 1.5pt),  
)
```

Preview



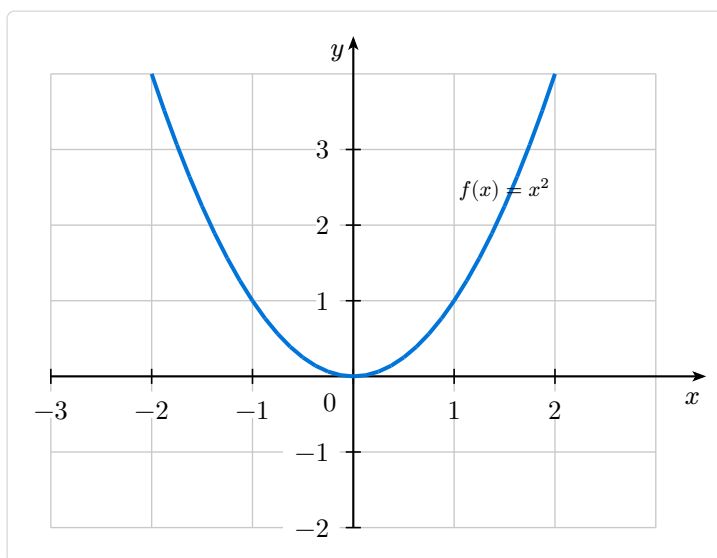
2.4. Function Labels

Add labels to your functions:

Code

```
#plot(  
  width: 8, height: 6,  
  xmin: -3, xmax: 3, ymin: -2, ymax: 4,  
  xlabel: $x$, ylabel: $y$,  
  show-grid: "major",  
  (  
    fn: x => x * x,  
    stroke: blue + 1.5pt,  
    label: $f(x) = x^2$,  
    label-side: "above",  
    label-pos: 0.75,  
  ),  
)
```

Preview



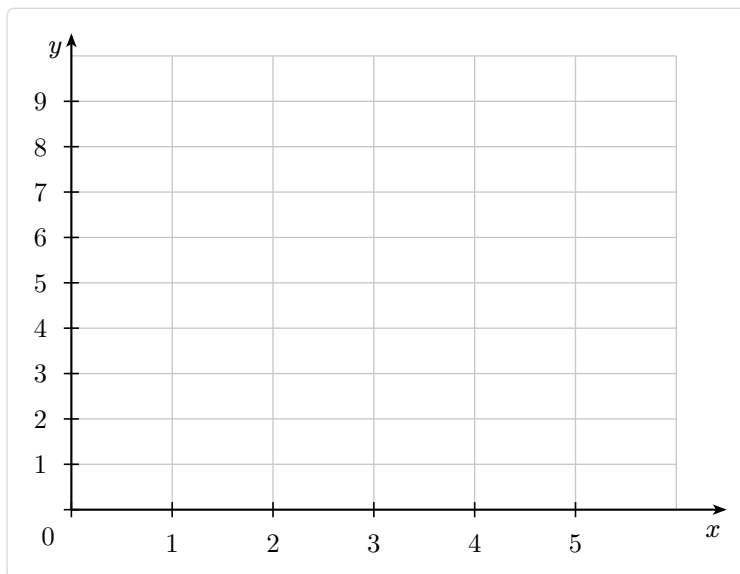
2.5. Data Points and Scatter Plots

Plot discrete data points:

Code

```
#plot(
  width: 8, height: 6,
  xmin: 0, xmax: 6, ymin: 0, ymax: 10,
  xlabel:  $x$ , ylabel:  $y$ ,
  axis-x-pos: "bottom", axis-y-pos: "left",
  show-grid: "major",
  (
    data: ((1, 2), (2, 4), (3, 5), (4, 7), (5, 9)),
    mark: "o",
    mark-size: 0.15,
    stroke: blue + 1pt,
  ),
)
```

Preview



3. Grid Options

3.1. Grid Modes

Control grid display with `show-grid`:

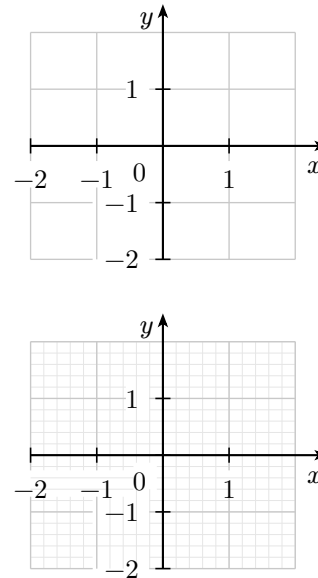
Code

```
// Major grid only
#plot(
  width: 5, height: 4,
  xmin: -2, xmax: 2, ymin: -2, ymax: 2,
  show-grid: "major",
)

// Minor grid only
#plot(
  width: 5, height: 4,
  xmin: -2, xmax: 2, ymin: -2, ymax: 2,
  show-grid: "minor",
)

// Both grids
#plot(
  width: 5, height: 4,
  xmin: -2, xmax: 2, ymin: -2, ymax: 2,
  show-grid: "both",
)
```

Preview



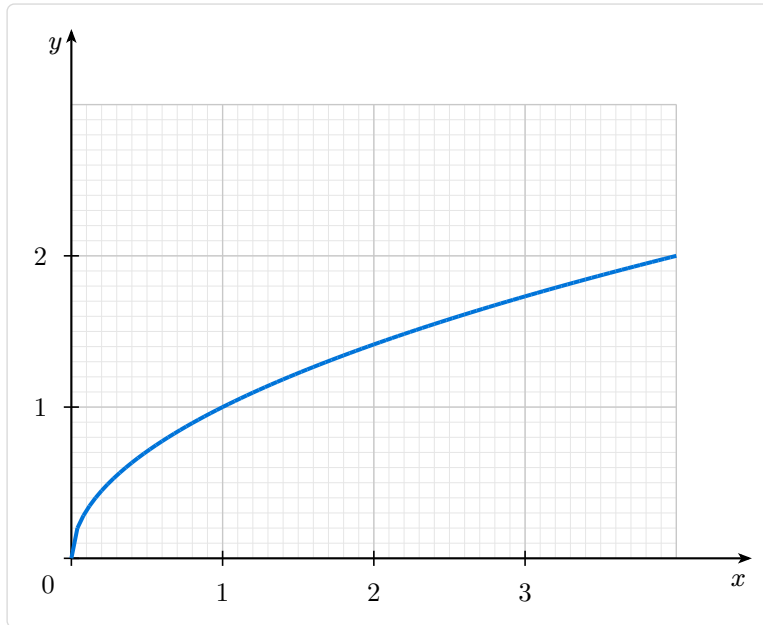
3.2. Minor Grid Subdivisions

Control the number of subdivisions with `minor-grid-step`:

Code

```
#plot(  
  width: 8, height: 6,  
  xmin: 0, xmax: 4, ymin: 0, ymax: 3,  
  xlabel:  $x$ , ylabel:  $y$ ,  
  axis-x-pos: "bottom", axis-y-pos: "left",  
  show-grid: "both",  
  minor-grid-step: 10, // 10 subdivisions per unit  
  (fn: x => calc.sqrt(x), domain: (0, 4), stroke: blue + 1.5pt),  
)
```

Preview



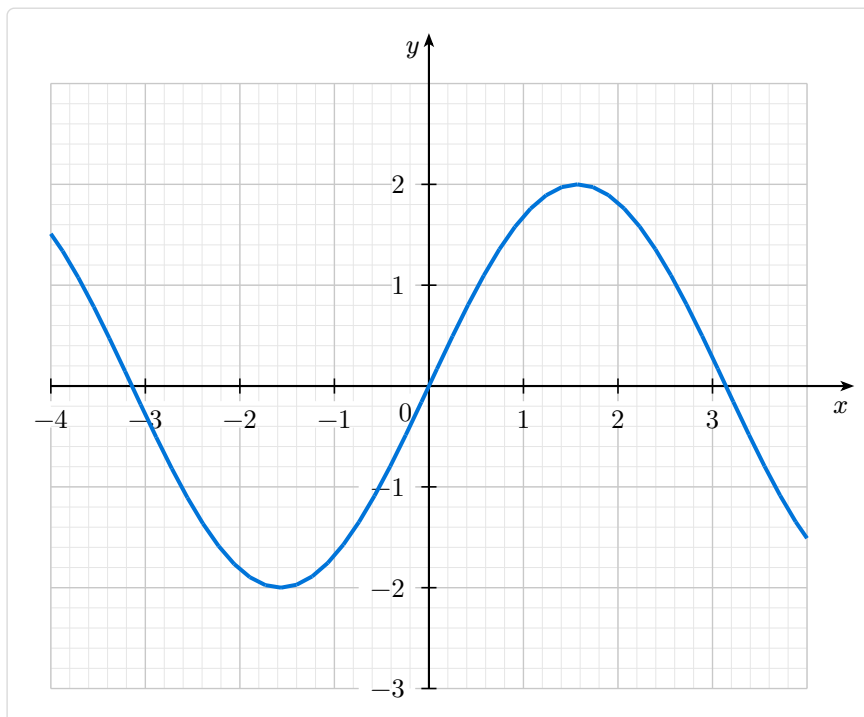
3.3. Grid Label Break

The `grid-label-break` option (enabled by default) draws grid lines with gaps around tick labels, creating an elegant break effect. Unlike a white-box approach, this works on any background color:

Code

```
#plot(  
  width: 10, height: 8,  
  xmin: -4, xmax: 4, ymin: -3, ymax: 3,  
  xlabel:  $x$ , ylabel:  $y$ ,  
  show-grid: "both",  
  minor-grid-step: 5,  
  grid-label-break: true, // Default  
  (fn: x => calc.sin(x) * 2, stroke: blue + 1.5pt),  
)
```

Preview



4. Axis Configuration

4.1. Axis Position

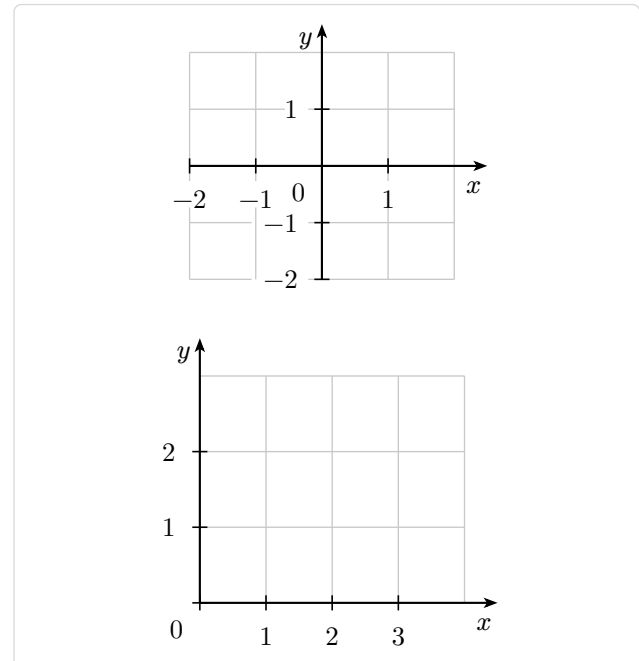
Position axes at origin (default), bottom/left, or custom values:

Code

```
// Through origin (default)
#plot(
  width: 5, height: 4,
  xmin: -2, xmax: 2, ymin: -2, ymax: 2,
  show-grid: "major",
)

// Bottom and left
#plot(
  width: 5, height: 4,
  xmin: 0, xmax: 4, ymin: 0, ymax: 3,
  axis-x-pos: "bottom",
  axis-y-pos: "left",
  show-grid: "major",
)
```

Preview



4.2. Axis Extension

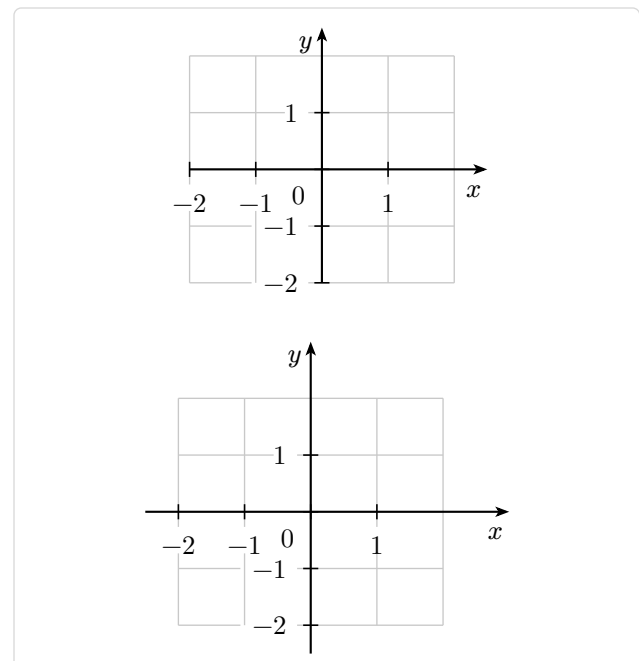
By default, axes extend 0.5 units beyond the grid on the arrow side. Customize with `axis-x-extend` and `axis-y-extend`:

Code

```
// Default extension (0, 0.5)
#plot(
  width: 5, height: 4,
  xmin: -2, xmax: 2, ymin: -2, ymax: 2,
  show-grid: "major",
)

// Custom extension
#plot(
  width: 5, height: 4,
  xmin: -2, xmax: 2, ymin: -2, ymax: 2,
  axis-x-extend: (0.5, 1),
  axis-y-extend: (0.5, 1),
  show-grid: "major",
)
```

Preview



5. Tick Configuration

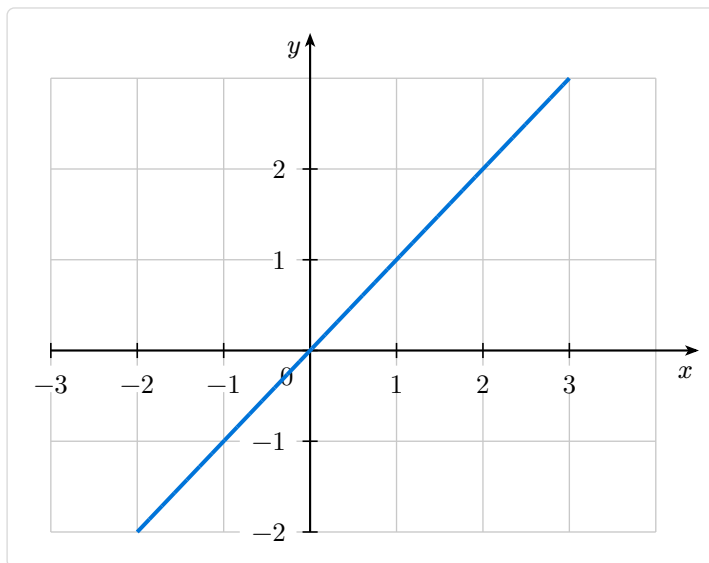
5.1. Default Integer Ticks

By default, ticks are placed at every integer (step = 1):

Code

```
#plot(  
  width: 8, height: 6,  
  xmin: -3, xmax: 4, ymin: -2, ymax: 3,  
  xlabel:  $x$ , ylabel:  $y$ ,  
  show-grid: "major",  
  (fn: x => x, stroke: blue + 1.5pt),  
)
```

Preview



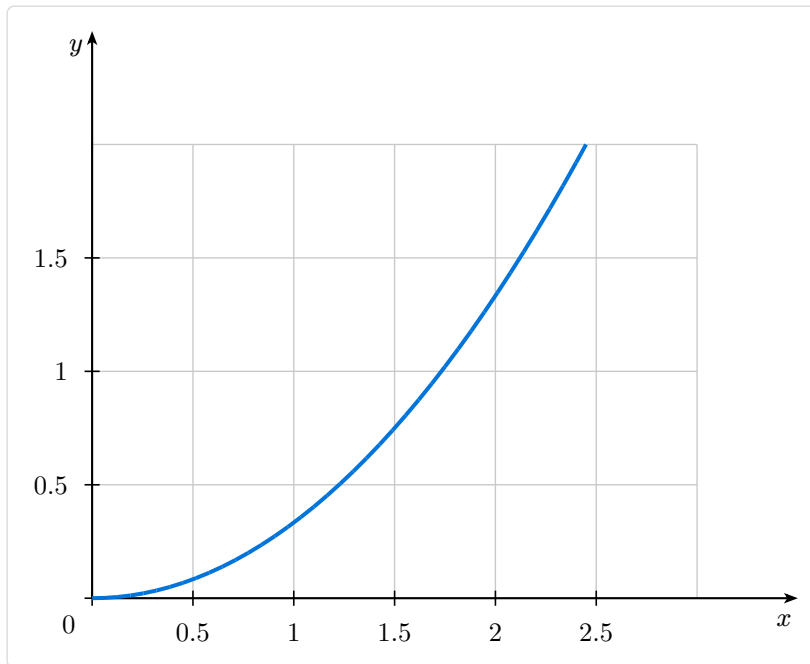
5.2. Custom Tick Step

Change tick spacing with `xtick-step` and `ytick-step`:

Code

```
#plot(  
  width: 8, height: 6,  
  xmin: 0, xmax: 3, ymin: 0, ymax: 2,  
  xlabel:  $x$ , ylabel:  $y$ ,  
  axis-x-pos: "bottom", axis-y-pos: "left",  
  xtick-step: 0.5,  
  ytick-step: 0.5,  
  show-grid: "major",  
  (fn: x => x * x / 3, stroke: blue + 1.5pt),  
)
```

Preview



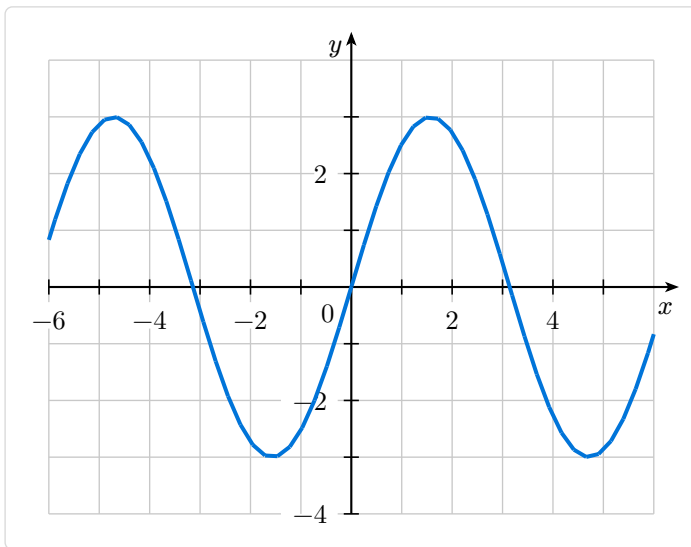
5.3. Tick Label Step

Show labels only at every N-th tick with `xtick-label-step` and `ytick-label-step`:

Code

```
#plot(
  width: 8, height: 6,
  xmin: -6, xmax: 6, ymin: -4, ymax: 4,
  xlabel:  $x$ , ylabel:  $y$ ,
  xtick-label-step: 2, // Labels at -6, -4, -2, 2, 4, 6
  ytick-label-step: 2, // Labels at -4, -2, 2, 4
  show-grid: "major",
  (fn: x => calc.sin(x) * 3, stroke: blue + 1.5pt),
)
```

Preview



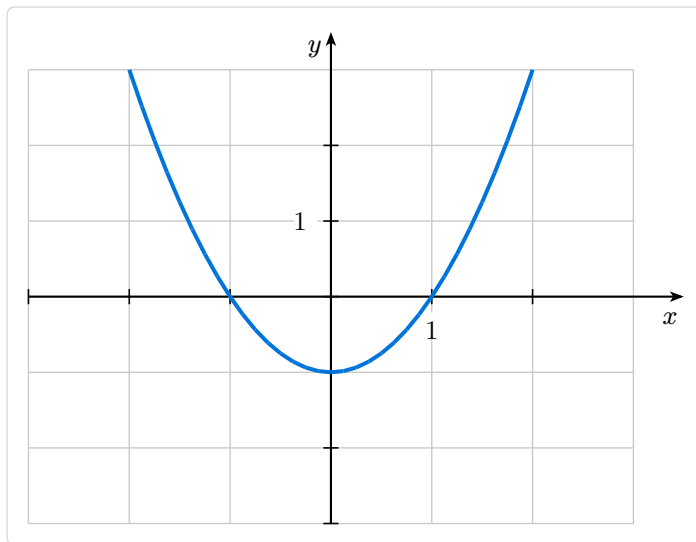
5.4. Unit Label Only

Show only “1” on each axis for a minimal style:

Code

```
#plot(  
  width: 8, height: 6,  
  xmin: -3, xmax: 3, ymin: -3, ymax: 3,  
  xlabel:  $x$ , ylabel:  $y$ ,  
  unit-label-only: true,  
  show-origin: false,  
  show-grid: "major",  
  (fn: x => x * x - 1, stroke: blue + 1.5pt),  
)
```

Preview



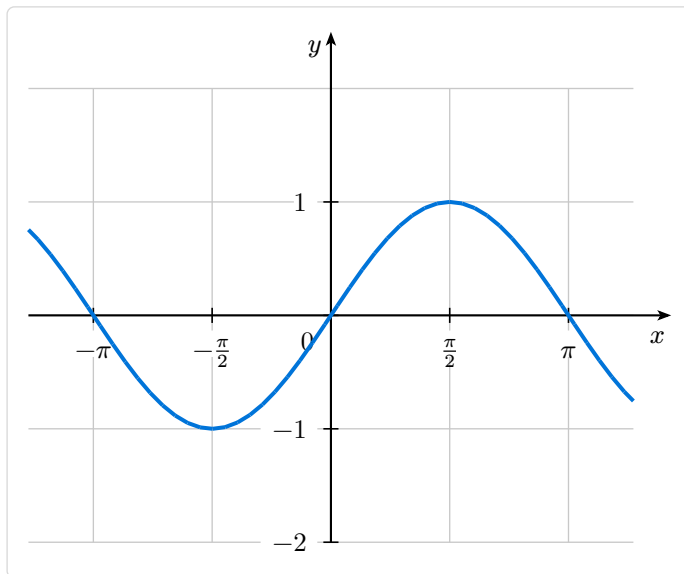
5.5. Custom Tick Positions

Specify exact tick positions with `xtick` and `ytick`:

Code

```
#plot(
  width: 8, height: 6,
  xmin: -4, xmax: 4, ymin: -2, ymax: 2,
  xlabel:  $x$ , ylabel:  $y$ ,
  xtick: (-calc.pi, -calc.pi/2, 0, calc.pi/2, calc.pi),
  xtick-labels: ( $-\pi$ ,  $-\pi/2$ ,  $0$ ,  $\pi/2$ ,  $\pi$ ),
  show-grid: "major",
  (fn: x => calc.sin(x), stroke: blue + 1.5pt),
)
```

Preview



5.6. Hide Origin Label

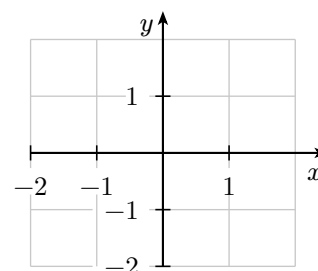
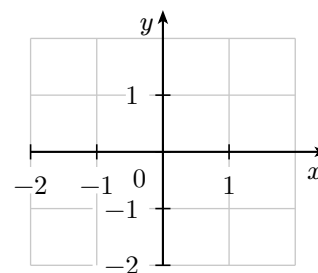
Control the “0” label at the origin:

Code

```
// With origin (default)
#plot(
  width: 5, height: 4,
  xmin: -2, xmax: 2, ymin: -2, ymax: 2,
  show-origin: true,
  show-grid: "major",
)

// Without origin
#plot(
  width: 5, height: 4,
  xmin: -2, xmax: 2, ymin: -2, ymax: 2,
  show-origin: false,
  show-grid: "major",
)
```

Preview



6. Markers

6.1. Available Marker Types

The following markers are available:

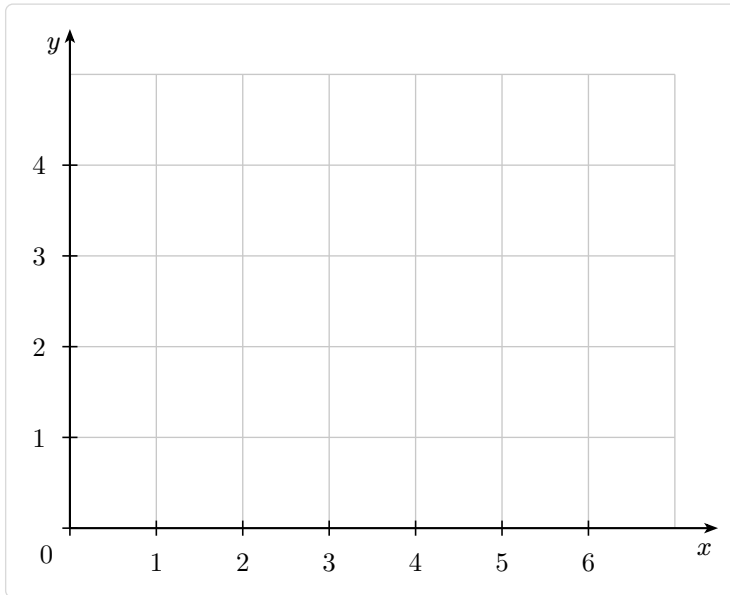
Marker	Description
"o"	Circle (outline)
"*"	Circle (filled)
"square" / "s"	Square (outline)
"square*"	Square (filled)
"triangle" / "^"	Triangle up (outline)
"triangle*"	Triangle up (filled)
"diamond" / "d"	Diamond (outline)
"diamond*"	Diamond (filled)
"star"	Star (outline)
"star*"	Star (filled)
"+"	Plus sign
"x"	Cross
" "	Vertical bar
"_"	Horizontal bar

6.2. Using Markers

Code

```
#plot(  
  width: 8, height: 6,  
  xmin: 0, xmax: 7, ymin: 0, ymax: 5,  
  axis-x-pos: "bottom", axis-y-pos: "left",  
  show-grid: "major",  
  (data: ((1, 1), (2, 2), (3, 2.5)), mark: "o", stroke: blue),  
  (data: ((1, 2), (2, 3), (3, 3.5)), mark: "square*", stroke: red),  
  (data: ((1, 3), (2, 4), (3, 4.2)), mark: "triangle", stroke: green),  
)
```

Preview



7. Convenience Functions

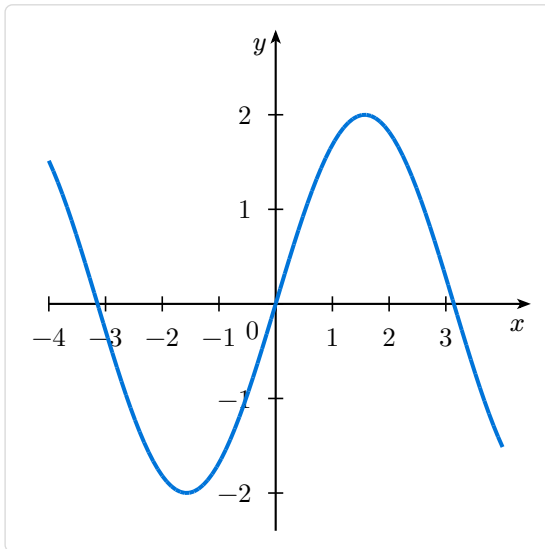
7.1. plot-fn - Quick Function Plot

Plot a single function with automatic y-scaling:

Code

```
#plot-fn(  
  x => calc.sin(x) * 2.0,  
  domain: (-4, 4),  
  stroke: blue + 1.5pt,  
)
```

Preview



7.2. scatter - Scatter Plot Helper

Create scatter plot specifications:

```
#let my-data = scatter(  
  ((1, 2), (2, 4), (3, 5)),  
  mark: "o",  
  stroke: blue,  
)  
#plot(xmin: 0, xmax: 4, ymin: 0, ymax: 6, my-data)
```

7.3. line-plot - Line Plot Helper

Create connected line plots:

```
#let my-line = line-plot(  
  ((1, 2), (2, 4), (3, 5)),  
  stroke: blue + 1pt,  
  mark: "o",  
)  
#plot(xmin: 0, xmax: 4, ymin: 0, ymax: 6, my-line)
```

7.4. func-plot - Function Plot Helper

Create function plot specifications:

```
#let my-func = func-plot(  
  x => calc.sin(x),  
  stroke: blue + 1.5pt,  
  label: $sin(x)$,  
)  
#plot(xmin: -4, xmax: 4, ymin: -1.5, ymax: 1.5, my-func)
```

7.5. riemann-sum - Riemann Sum Rectangles

Draw left, right, midpoint, lower, or upper Riemann sum rectangles for a function. Use it as a plot item alongside the function curve or fill-area.

The five methods:

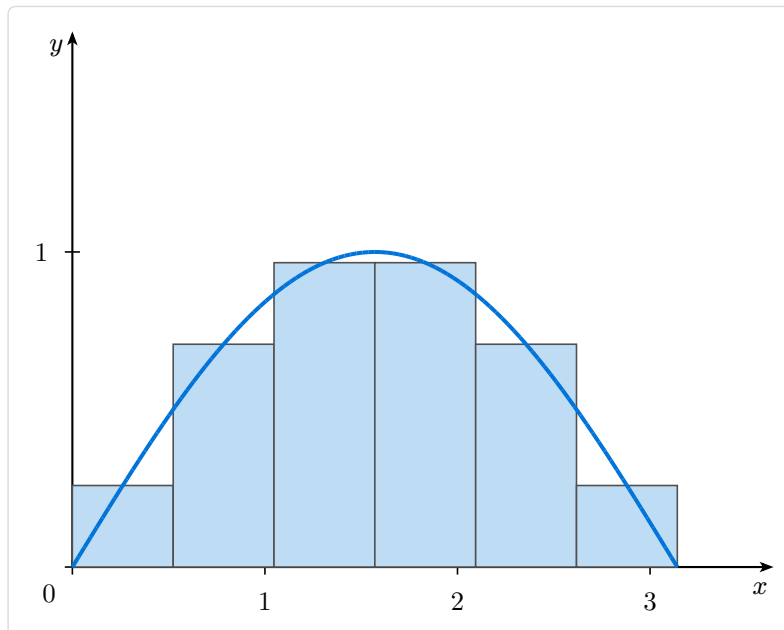
- "left" / "right" / "mid" — evaluation at the left endpoint, right endpoint, or midpoint of each sub-interval
- "lower" / "upper" — true infimum / supremum sampled within each sub-interval; correct for any function shape including U-curves

7.5.1. Basic Example

Code

```
#plot(
  width: 8, height: 5,
  xmin: 0, xmax: calc.pi, ymin: 0, ymax: 1.2,
  axis-x-pos: "bottom", axis-y-pos: "left",
  riemann-sum(
    x => calc.sin(x),
    domain: (0.0, calc.pi),
    n: 6, method: "mid",
    color: blue.lighten(75%),
  ),
  (fn: x => calc.sin(x), domain: (0.0, calc.pi), stroke: blue + 1.5pt),
)
```

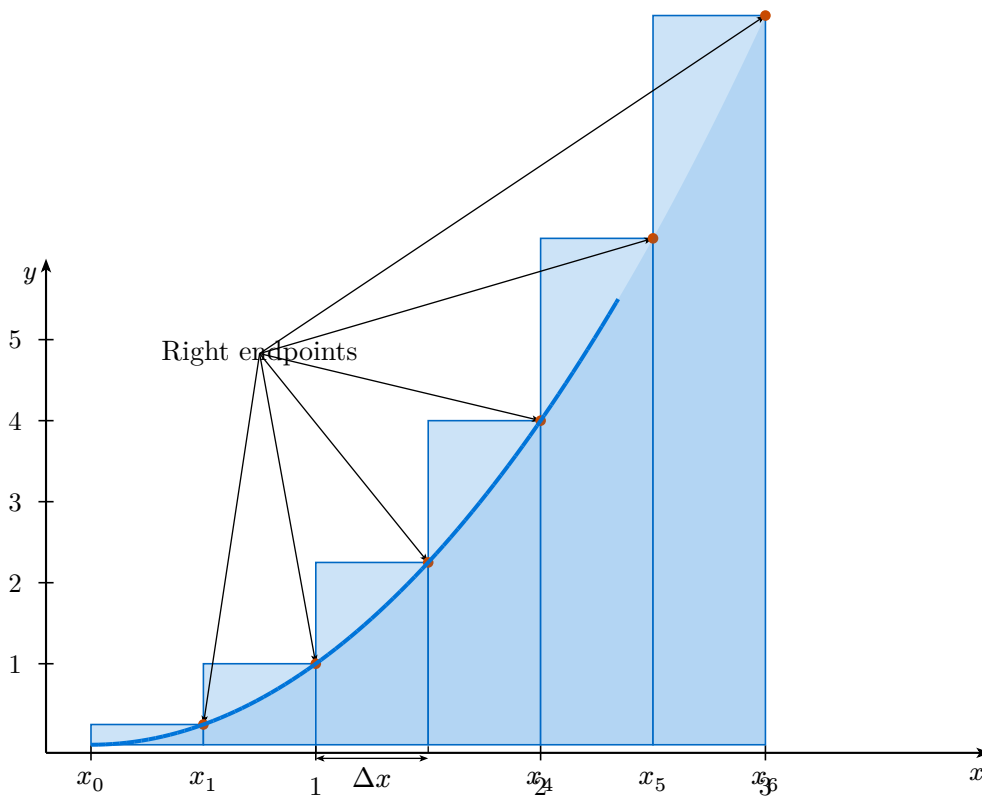
Preview



7.5.2. Annotations: show-points, show-dx, show-xi

Turn on the annotation overlays for pedagogical diagrams:

```
riemann-sum(
  x => x * x,
  domain: (0.0, 3.0),
  n: 6, method: "right",
  color: blue.lighten(80%),
  show-points: true,           // dots at evaluation points with arrows
  show-dx: true, dx-rect: 2,   // Δx bracket under rectangle 2
  show-xi: true,               // x₀ ... x₆ labels along the axis
)
```



7.5.3. xi-show-values

Set `xi-show-values: true` to display the actual numeric values of the subdivision points instead of the symbolic x_i notation:

```
riemann-sum(fn, domain: (0.0, 4.0), n: 4, method: "left",
            show-xi: true, xi-show-values: true)
```

7.5.4. Hatch patterns

Use `hatch` together with `hatch-spacing` and `hatch-stroke` to draw hatched rectangles. Set `color: none` for hatch-only (no fill):

```
riemann-sum(fn, domain: ..., n: 6, method: "mid",
            color: none,
            hatch: "ne", hatch-spacing: 4pt, hatch-stroke: blue + 0.6pt)
```

Available hatch styles: "ne", "nw", "h", "v", "cross", "grid".

7.6. volume-of-revolution - Volume of revolution diagram

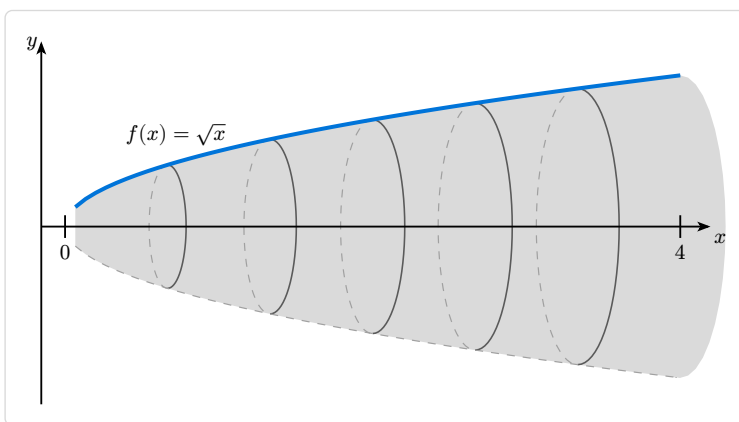
Draw a 3D-style solid generated by rotating a profile $y = f(x)$ around a horizontal or oblique axis. The renderer draws the filled body, front and back cap ellipses, intermediate disk cross-sections, axis arrow, coordinate y-axis, and optional labels.

7.6.1. Basic Example

Code

```
#volume-of-revolution(  
  x => calc.sqrt(x),  
  domain: (0.0, 4.0),  
  n-disks: 5,  
  width: 8.0, height: 4.0,  
  show-yaxis: true,  
  label-a: $0$, label-b: $4$,  
  label-f: $f(x)=\sqrt{x}$,  
)
```

Preview

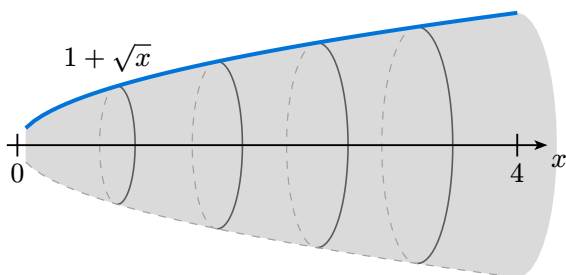


7.6.2. Axis of Revolution

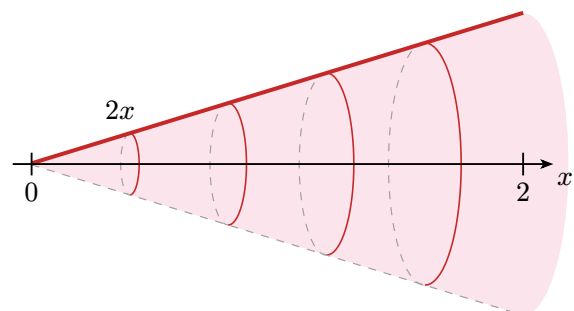
Use `axis-y` to revolve around $y = c$ (shifted horizontal axis), or `axis-slope` for an oblique axis $y = mx +$ `axis-y`:

```
// Around y = 1  
#volume-of-revolution(x => 1.0 + calc.sqrt(x), domain: (0, 4), axis-y: 1.0)  
  
// Around y = x (oblique)  
#volume-of-revolution(x => x * x, domain: (0.01, 2), axis-slope: 1.0)
```

Around $y = 1$



Around $y = x$ (oblique)



7.6.3. show-back and show-radius-marker

`show-back: false` hides the dashed back half and bottom profile — useful for side-by-side comparisons. `show-radius-marker: true` draws a vertical dimension marker at the y-axis:

```
#volume-of-revolution(fn, domain: ...,
  show-back: false,
  show-radius-marker: true,
  yaxis-x: 2.0,    // place y-axis at x=2, not the auto left position
  label-y:  $\sqrt{2}$ ,
)
```

7.6.4. show-y-axis and y-axis placement

The coordinate y-axis is positioned automatically to the left of the solid by default. Use `y-axis-x` / `yaxis-x` to fix it at a specific x-value, `y-axis-offset` to change the automatic gap, and `y-axis-extend` to control how far the axis extends above and below the solid:

```
#volume-of-revolution(fn, domain: ...,
  show-yaxis: true,
  y-axis-offset: 0.6,           // more gap from the volume
  y-axis-extend: (0.2, 0.5),   // tighter below, more above
)
```

8. Global Configuration

8.1. Setting Defaults

Use `set-plot-defaults` to configure defaults for all subsequent plots:

```
#set-plot-defaults(  
  width: 8,  
  height: 6,  
  show-grid: "both",  
  minor-grid-step: 5,  
)  
  
// All plots will now use these defaults  
#plot(xmin: -3, xmax: 3, ymin: -2, ymax: 2, ...)
```

8.2. Resetting Defaults

Reset to original defaults:

```
#reset-plot-defaults()
```

9. Styling

9.1. Custom Styles

Override default styles with the `style` parameter:

```
#plot(  
  xmin: -3, xmax: 3, ymin: -2, ymax: 2,  
  style: (  
    axis: (stroke: black + 1pt, arrow: "stealth"),  
    grid: (  
      major: (stroke: luma(180) + 0.6pt),  
      minor: (stroke: luma(220) + 0.3pt),  
    ),  
    ticks: (  
      length: 0.12,  
      stroke: black + 0.6pt,  
      label-size: 0.7em,  
    ),  
  ),  
  ...  
)
```

9.2. Default Style Values

Property	Default	Description
axis.stroke	black + 0.8pt	Axis line style
axis.arrow	"stealth"	Arrow head style
grid.major.stroke	luma(200) + 0.5pt	Major grid line style
grid.minor.stroke	luma(230) + 0.3pt	Minor grid line style
ticks.length	0.1	Tick mark length (cm)
ticks.stroke	black + 0.6pt	Tick mark style
ticks.label-size	0.65em	Tick label font size
ticks.label-offset	0.15	Distance from tick to label
plot.stroke	blue + 1.2pt	Default function stroke
plot.samples	100	Default sample count
marker.size	0.12	Default marker size
labels.size	0.8em	Axis label font size

10. Parameter Reference

10.1. plot Function

Dimensions and Bounds:

Parameter	Type	Default	Description
width	float	6	Plot width in cm
height	float	6	Plot height in cm
scale	float	1	Scale factor for entire plot
xmin	float	auto	Minimum x value
xmax	float	auto	Maximum x value
ymin	float	auto	Minimum y value
ymax	float	auto	Maximum y value

Axis Configuration:

Parameter	Type	Default	Description
xlabel	content	none	X-axis label
ylabel	content	none	Y-axis label
xlabel-pos	string/array	“end”	“end”, “center”, or (x, y)
ylabel-pos	string/array	“end”	“end”, “center”, or (x, y)
xlabel-anchor	string	“west”	Anchor for x label
ylabel-anchor	string	“south”	Anchor for y label
xlabel-offset	array	(0.3, 0)	X label offset (cm)
ylabel-offset	array	(0, 0.3)	Y label offset (cm)
axis-x-pos	string/float	0	“bottom”, “center”, or y-value
axis-y-pos	string/float	0	“left”, “center”, or x-value
axis-x-extend	float/array	(0, 0.5)	X-axis extension (left, right)
axis-y-extend	float/array	(0, 0.5)	Y-axis extension (bottom, top)

Tick Configuration:

Parameter	Type	Default	Description
xtick	array/none	auto	Custom x tick positions
ytick	array/none	auto	Custom y tick positions
xtick-step	float	1	X tick spacing
ytick-step	float	1	Y tick spacing
xtick-labels	array/none	auto	Custom x tick labels
ytick-labels	array/none	auto	Custom y tick labels
xtick-label-step	int	1	Show x label every N ticks
ytick-label-step	int	1	Show y label every N ticks

show-origin	bool	true	Show “0” at origin
unit-label-only	bool	false	Show only “1” on axes
tick-label-size	length	0.65em	Tick label font size
axis-label-size	length	0.8em	Axis label font size

Grid Configuration:

Parameter	Type	Default	Description
show-grid	bool/string	false	true, false, “major”, “minor”, “both”
minor-grid-step	int	5	Subdivisions per major tick
grid-label-break	bool	true	Break grid lines around labels

Styling:

Parameter	Type	Default	Description
style	dictionary	none	Style overrides

10.2. Function/Data Specification

Each plot item is a dictionary with these fields:

Field	Type	Description
fn	function	Function to plot: $x \Rightarrow y$
data	array	Data points: $((x_1, y_1), (x_2, y_2), \dots)$
domain	array	Function domain: $(x_{\min}, x_{\max})$
samples	int	Number of samples for function
stroke	stroke	Line style
mark	string	Marker type
mark-size	float	Marker size in cm
mark-fill	color	Marker fill color
label	content	Label text
label-pos	string	“above”, “below”, “left”, “right”
label-at	float/string	x-position or “start”/“end”/“center”
label-anchor	string	Text anchor point

10.3. riemann-sum Function

Parameter	Type	Default	Description
fn	function	—	Function $f(x)$ to approximate
domain	array	plot range	Interval (a , b); overrides the plot's x-range
n	int	4	Number of rectangles
method	string	"right"	"left", "right", "mid", "lower", "upper"
baseline	float	0.0	Y-level of rectangle bases (default: x-axis)
color	color	luma(220)	Rectangle fill; none for hatch-only
stroke	stroke	luma(80) + 0.6pt	Rectangle border stroke
hatch	string/none	none	Hatch pattern: "ne", "nw", "h", "v", "cross", "grid"
hatch-spacing	length	5pt	Spacing between hatch lines
hatch-stroke	stroke	luma(80) + 0.5pt	Stroke for hatch lines
samples	int	20	Samples per sub-interval for "lower" / "upper" methods
show-points	bool	false	Draw a dot at each evaluation point
point-color	color	rgb("#c94a00")	Dot fill color
point-size	float	0.07	Dot radius in cm
point-label	content/auto/none	auto	Arrow label near dots; auto = method name, none = dots only
point-label-pos	array/auto	auto	(x , y) in data coords; auto = upper-right of dots
show-dx	bool	false	Draw a Δx dimension bracket under one rectangle
dx-rect	int/auto	auto	Rectangle to annotate (0-based); auto = middle rectangle
dx-label	content	Δx	Label displayed inside the bracket
show-xi	bool	false	Draw $x_0, x_1, \dots, x_n$ at subdivision points
xi-labels	array/auto	auto	Custom label array; auto = subscripted x_i notation
xi-show-values	bool	false	Show numeric x-values instead of x_i notation

10.4. volume-of-revolution Function

solid-of-revolution is an alias kept for backward compatibility.

Parameter	Type	Default	Description
fn	function	—	Profile function $f(x)$; the curve that is revolved
domain	array	(0.0, 4.0)	Interval (a, b) — the revolution interval
n-disks	int	4	Number of intermediate disk cross-sections to show
width / height	float	5.0 / 3.5	Canvas size in cm
samples	int	60	Number of points sampled along the profile curve
axis-y	float	0.0	Y-value of horizontal revolution axis ($y = c$)
axis-slope	float	0.0	Slope m : revolution axis is $y = mx + \text{axis-y}$
show-axis	bool	true	Draw the revolution axis with an arrowhead
show-y-axis / show-yaxis	bool	false	Draw a coordinate y-axis at the left cap
y-axis-x / yaxis-x	float/auto	auto	X-position for the y-axis; auto = left of volume
y-axis-offset	float	0.45	Gap between y-axis and the volume when auto
y-axis-extend	array	(0.35, 0.45)	Y-axis padding (below, above) the solid
show-radius-marker	bool	false	Draw a vertical radius dimension marker at yaxis-x
show-back	bool	true	Show the back half of the solid (dashed ellipses + bottom curve)
show-labels	bool	true	Show a , b , f labels
profile-stroke	stroke	blue + 1.5pt	Top profile curve stroke
disk-color	color	luma(218)	Solid body fill color
disk-stroke	stroke	luma(90) + 0.6pt	Disk edge stroke
axis-stroke	stroke	black + 0.7pt	Revolution axis stroke
label-a / label-b	content	$\$a\$$ / $\$b\$$	Labels at domain endpoints
label-f	content	$\$f\$$	Function label near the profile curve
label-y	content	$\$y\$$	Label for the coordinate y-axis

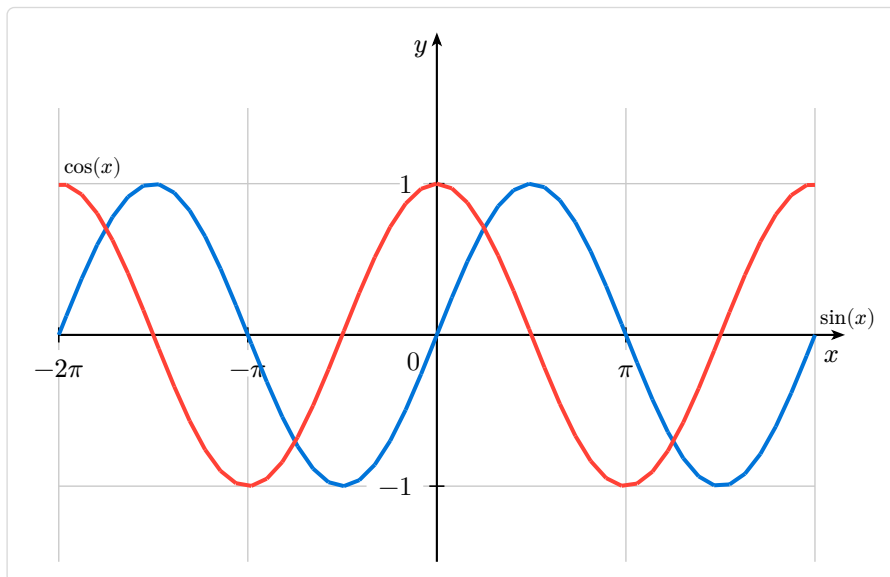
11. Complete Examples

11.1. Trigonometric Functions

Code

```
#plot(  
  width: 10, height: 6,  
  xmin: -2 * calc.pi, xmax: 2 * calc.pi,  
  ymin: -1.5, ymax: 1.5,  
  xlabel:  $x$ , ylabel:  $y$ ,  
  xtick: (-2*calc.pi, -calc.pi, 0, calc.pi, 2*calc.pi),  
  xtick-labels: ( $-2\pi$ ,  $-\pi$ ,  $0$ ,  $\pi$ ,  $2\pi$ ),  
  show-grid: "major",  
  (fn: x => calc.sin(x), stroke: blue + 1.5pt, label:  $\sin(x)$ , label-pos: 1),  
  (fn: x => calc.cos(x), stroke: red + 1.5pt, label:  $\cos(x)$ , label-pos: 0),  
)
```

Preview

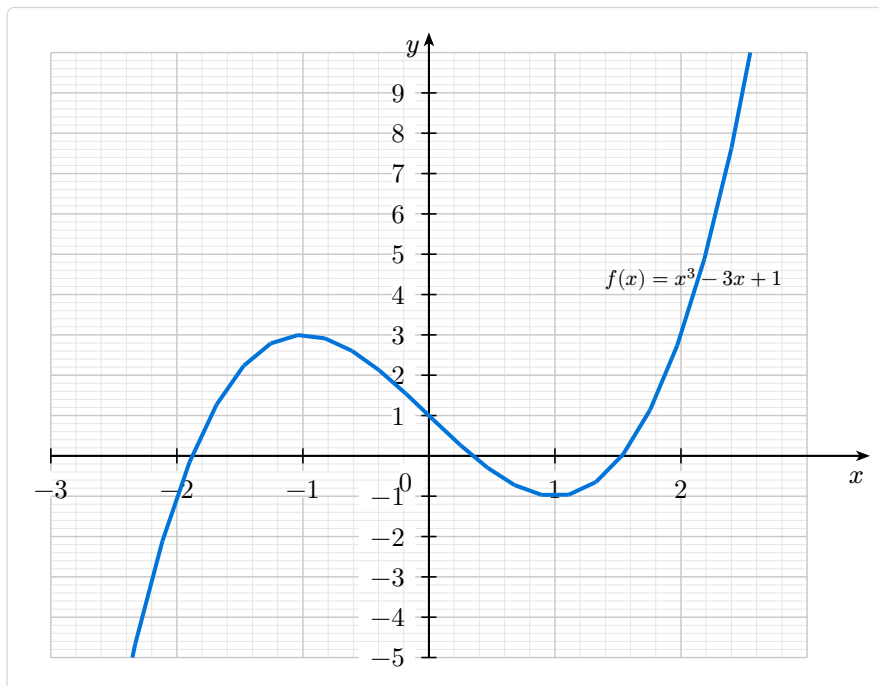


11.2. Polynomial with Fine Grid

Code

```
#plot(  
  width: 10, height: 8,  
  xmin: -3, xmax: 3, ymin: -5, ymax: 10,  
  xlabel: $x$, ylabel: $y$,  
  show-grid: "both",  
  minor-grid-step: 5,  
  (  
    fn: x => x * x * x - 3 * x + 1,  
    stroke: blue + 1.5pt,  
    label: $f(x) = x^3 - 3x + 1$,  
    label-side: "above",  
    label-pos: 0.85,  
  ),  
)
```

Preview

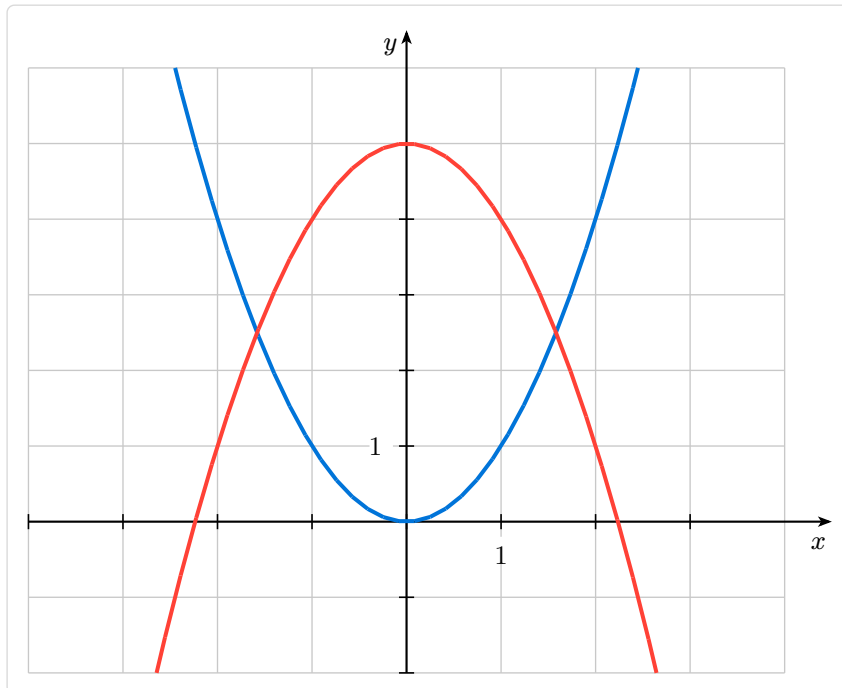


11.3. Minimal Style Plot

Code

```
#plot(  
  width: 10, height: 8,  
  xmin: -4, xmax: 4, ymin: -2, ymax: 6,  
  xlabel:  $x$ , ylabel:  $y$ ,  
  show-grid: "major",  
  unit-label-only: true,  
  show-origin: false,  
  (fn: x => x * x, stroke: blue + 1.5pt),  
  (fn: x => -x * x + 5, stroke: red + 1.5pt),  
)
```

Preview



11.4. Data with Trend Line

Code

```
#plot(
  width: 10, height: 7,
  xmin: 0, xmax: 6, ymin: 0, ymax: 12,
  xlabel: "Time (s)", ylabel: "Distance (m)",
  axis-x-pos: "bottom", axis-y-pos: "left",
  show-grid: "both",
  minor-grid-step: 5,
  (fn: x => 2 * x, stroke: gray + 1pt, domain: (0, 6)), // Trend line
  (
    data: ((0.5, 1.2), (1, 2.3), (2, 3.8), (3, 6.2), (4, 7.9), (5, 10.1)),
    mark: "o",
    mark-size: 0.12,
    stroke: none,
  ),
)
```

Preview

